JETBRAINS | Kotlin

Parallel & Concurrent Programming

kotlinglang.org/education



Definition

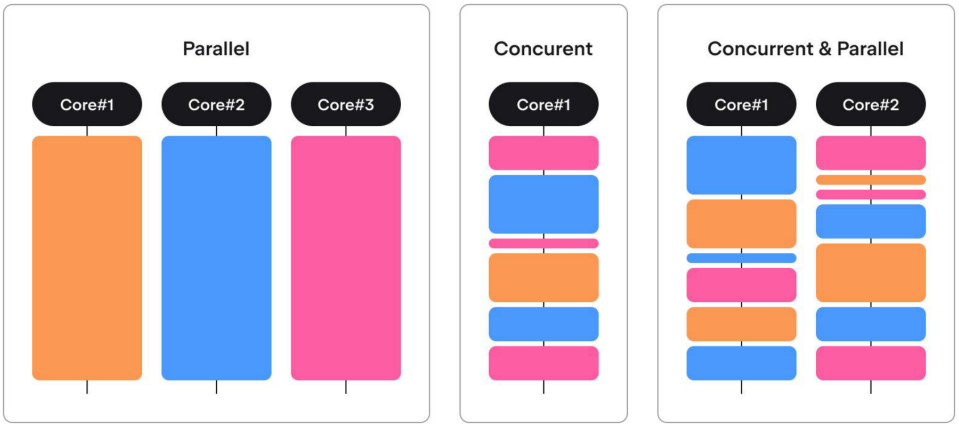
According to Wikipedia:

- **Parallel** computing is a type of computing “in which many calculations or processes are carried out **simultaneously**”.
- **Concurrent** computing is a form of computing in which several computations are executed **concurrently –** in overlapping time periods – instead of sequentially.
- It is possible to have parallelism without concurrency, and concurrency without parallelism.

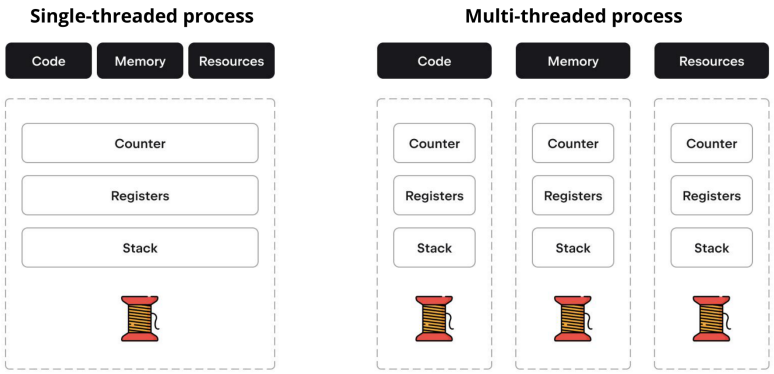
Motivation

- Faster runtime
- Improved responsiveness

Parallelism vs concurrency

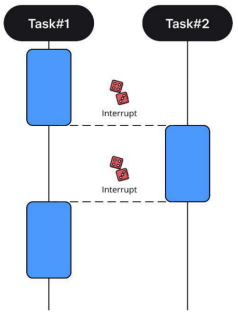


Concurrency: processes vs threads

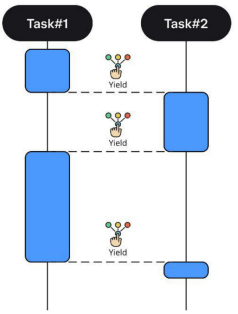


Preemptive vs cooperative scheduling

Preemptive: OS interrupts tasks



Cooperative: task yields control



Parallel and concurrent Programming in the JVM

- The JVM has its own scheduler
  - It is independent from the OS scheduler
  - A JVM thread ≠ an OS thread
  - ⇒ Multithreaded JVM apps can run on a single-threaded OS
- (DOS) JVM threads are either daemons or user threads.
- The app stops when all **user threads** are done.
- The JVM does not wait for daemon threads to finish.

Parallel programming in the JVM

2 Java packages

- `java.lang` contains basic primitives: `Runnable`, `Thread`, etc
- `java.util.concurrent` contains synchronization primitives and concurrent data structures

Kotlin package

- `kotlin.concurrent` — Wrappers and extensions for Java classes

Throwback: Single abstract method interfaces

```java
@FunctionalInterface
public interface Runnable {
   public abstract void run();
}
```

Interface with a single method. We can instantiate it with a lambda.

```
class RunnableWrapper(val runnable: Runnable)

val myWrapperObject =
   RunnableWrapper(
       object : Runnable {
           override fun run() {
               println("I run")
           }
       }
   )
val myWrapperLambda = RunnableWrapper
{ println("yo") }
```

Ways to create threads

You can inherit from the `Thread` class, which also implements `Runnable`.

```
class MyThread : Thread() {
    override fun run() {
        println("${currentThread()} is
running")
    }
}

fun main() {
    val myThread = MyThread()
    myThread.start()
}
```

run vs start

Never call Thread.run()! `run` will execute on *your* thread, while `start` will create a new thread where `run` will be executed.

```
fun main() {
    val myThread1 = MyThread()
    myThread1.start() // OK
    val myThread2 = MyThread()
    myThread2.run() // Current thread gets
blocked
}
```

Ways to create threads

You can implement the `Runnable` interface and pass it to a thread. You can pass the same `Runnable` to several threads.

```
fun main() {
    val myRunnable = Runnable
{ println("Sorry, gotta run!")
}
    val thread1 = Thread(myRunnable)
    thread1.start()
    val thread2 = Thread(myRunnable)
    thread2.start()
}
```

Ways to create threads

Kotlin has an even simpler way to create threads, but under the hood the same old thread is created and started.

```
import kotlin.concurrent.thread

fun main() {
    val kotlinThread = thread {

println("I start instantly, but you can
pass an option to start me later")
    }
}
```

This is the preferable way to create threads.

Thread properties

A thread's properties cannot be changed after it is started.

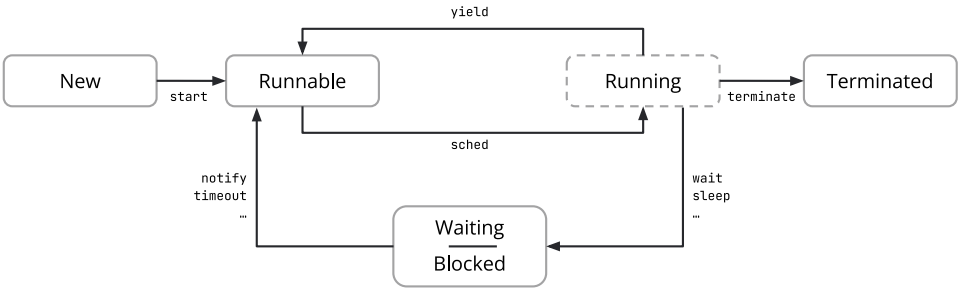
Main properties of a thread:

- `id: Long` — This is the thread's identifier
- `name: String`
- `priority: Int` — This can range from 1 to 10, with a larger value indicating higher priority
- `daemon: Boolean`
- `state: Thread.state`
- `isAlive: Boolean`

State of a thread

state	isAlive
NEW	false
RUNNABLE	true
BLOCKED	true
WAITING	true
TIMED_WAITING	true
TERMINATED	false

State of a thread



Ways to manipulate a thread's state

- `val myThread = thread { ... }` — Creates a new thread
- `myThread.start()` — Starts a `thread`
- `myThread.join()` — Causes the current thread to wait for another thread to finish
- `sleep(...)` — Puts the current thread to sleep
- `yield()` — *Tries* to step back
- `myThread.interrupt()` — *Tries* to interrupt a thread
- `myThread.isInterrupted()` — Checks whether thread was interrupted
- `interrupted()` — Checks and clears the interruption flag

sleep, join, yield, interrupt

- The `sleep` and `yield` methods are only applicable to the current thread, which means that you cannot suspend another thread.
- All blocking and waiting methods can throw `InterruptedException`

Classic worker

```
class ClassicWorker : Runnable {
   override fun run() {
       try {
           while (!Thread.interrupted()) {
               // do stuff
           }
       } catch (e: InterruptedException) {} //
absolutely legal empty catch block
   }
}
```

Parallelism and shared memory: Examples of problematic interleaving

Parallel threads have access to the same shared memory. This often leads to problems that cannot arise in a single-threaded environment.

```
class Counter {
    private var c = 0

    fun increment() {
        c++
    }
    fun decrement() {
        c--
    }
    fun value(): Int {
        return c
    }
}
```

Both operations on `c` are single, simple statements.

However, even simple statements can be translated into multiple steps by the virtual machine, and those steps can be interleaved.

Parallelism and shared memory: Examples of problematic interleaving

Parallel threads have access to the same shared memory. This often leads to problems that cannot arise in a single-threaded environment.

```
class Counter {
   private var c = 0

   fun increment() {
       c++
   }
   fun decrement() {
       c--
   }
   fun value(): Int {
       return c
   }
}
```

Suppose both Thread#1 and Thread#2 invoke increment at the same time. If the initial value of `c` is `0`, their interleaved actions might follow this sequence:

- T#1: Read value `0` from `c`.
- T#2: Read value `0` from `c`.
- T#1: Increment value — result is `1`.
- T#1: Write result `1` to `c`.
- T#2: Increment value — result is `1`.
- T#2: Write result `1` to `c`.

Synchronization mechanisms

- Mutual exclusion, such as `Lock` and the `synchronized` keyword
- Concurrent data structures and synchronization primitives
- Atomics, which work directly with shared memory (DANGER ZONE)
-

Locks

```
class LockedCounter {

   private var c = 0

   private val lock = ReentrantLock()

   fun increment() {
       lock.withLock { c++ }
   }

   // same for other methods
   …

}
```

The lock interface

- `lock.lock()` — Acquires the lock
- `lock.tryLock()` — Tries to acquire the lock
- `lock.unlock()` — Releases the lock
- `lock.withLock { }` — Executes a lambda with the lock held (has try/catch inside)
- `lock.newCondition()` — Creates a *condition variable* associated with the lock

Conditions

```
class PositiveLockedCounter {
   private var c = 0
   private val lock = ReentrantLock()
   private val condition = lock.newCondition()

   fun increment() {
       lock.withLock {
           c++
           condition.signal()
       }
   }

   fun decrement() {
       lock.withLock {
           while (c == 0) {
               condition.await()
           }
           c--
       }
   }

   fun value(): Int {
       return lock.withLock { c }
   }
}
```

A condition allows a thread holding a lock to *wait* until another thread *signals* it about a certain event. Internally, the `await` method releases the associated lock upon call, and acquires it back before finally returning it again.

The ReentrantLock class

- `ReentrantLock` – Allows the lock to be acquired multiple times by the same thread
- `lock.getHoldCount()` – Gets the number of holds on this lock by the current thread
- `lock.queuedThreads()` – Gets a collection of the threads waiting on this lock
- `lock.isFair()` – Checks the *fairness* of the lock

The synchronized statement

In the JVM, every object has an *intrinsic* lock associated with it (aka a *monitor*).

```
class Counter {
   private var c = 0

   fun increment() {
       synchronized(this) { c++ }
   }

   …
}
```

Synchronized method

Java

```java
public class SynchronizedCounter {
   private int c = 0;

   public synchronized void increment() {
      c++;
   }

   …
}
```

Kotlin

```kotlin
class SynchronizedCounter {
   private var c = 0

   @Synchronized
   fun increment() {
       c++
   }
   …
}
```

The ReadWriteLock class

`ReadWriteLock` allows multiple readers to access a resource concurrently but only lets a single writer modify it.

- `rwLock.readLock()` – Returns the read lock
- `rwLock.writeLock()` – Returns the write lock
- `rwLock.read { ... }` – Executes lambda under a read lock
- `rwLock.write { ... }` – Executes lambda under a write lock

The ReadWriteLock Class

```
class PositiveLockedCounter {
    private var c = 0
    private val rwLock = ReadWriteReentrantLock()

    fun increment() {
        rwLock.write { c++ }
    }

    fun decrement() {
        rwLock.write { c-- }
    }

    fun value(): Int {
        return rwLock.read { c }
    }
}
```

Concurrent blocking collections

`java.util.concurrent` is a Java package that implements both blocking and non-blocking concurrent collections, such as:

- `SynchronousQueue` – One-element rendezvous channel
- `ArrayBlockingQueue` – Fixed-capacity queue
- `LinkedBlockingQueue` – Unbounded blocking queue
- `PriorityBlockingQueue` – Unbounded blocking priority queue

Concurrent non-blocking collections

`java.util.concurrent` is a Java package that implements both blocking and non-blocking concurrent collections, such as:

- `ConcurrentLinkedQueue` – Non-blocking unbounded queue
- `ConcurrentLinkedDequeue` – Non-blocking unbounded dequeue
- `ConcurrentHashMap` – Concurrent unordered hash-map
- `ConcurrentSkipListMap` – Concurrent sorted hash-map

Synchronization primitives

`java.util.concurrent` also implements concurrent data structures and synchronization primitives.

- `Exchanger` – Blocking exchange
- `Phaser` – Barrier synchronization

Java Memory Model: Weak behaviors

There are no guarantees when it comes to ordering!

```
class OrderingTest {
    var x = 0
    var y = 0
    fun test() {
        thread {
            x = 1
            y = 1
        }
        thread {
            val a = y
            val b = x
            println("$a, $b")
        }
    }
}
```

Possible outputs:

- 0, 0
- 0, 1
- 1, 1
- 1, 0

Java Memory Model: Weak behaviors

There are no guarantees when it comes to progress!

```
class ProgressTest {
    var flag = false
    fun test() {
        thread {
            while (!flag) {}
            println("I am free!")
        }
        thread { flag = true }
    }
}
```

Possible outputs:

- "I am free!"
- ...
- ...
- ...
- hang!

Java Memory Model: Weak behaviors

There are no guarantees when it comes to progress!

```
class ProgressTest {
    var flag = false
    fun test() {
        thread {
            while (true) {}
            println("I am free!")
        }
        thread { flag = true }
    }
}
```

Possible outputs:

- "I am free!"
- ...
- ...
- ...
- hang!

JMM: Data-Race-Freedom Guarantee

But what does JMM guarantee?

Well-synchronized programs have **simple interleaving semantics.**

JMM: Data-Race-Freedom Guarantee

But what does JMM guarantee?

Well-synchronized programs have **simple interleaving semantics.**

Well-synchronized = Data-race-free

Simple interleaving semantics = Sequentially consistent semantics

Data-race-free programs have **sequentially consistent** semantics

JMM: Volatile fields

Volatile fields can be used to restore **sequential consistency**.

```
class OrderingTest {
    @Volatile var x = 0
    @Volatile var y = 0
    fun test() {
        thread {
            x = 1
            y = 1
        }
        thread {
            val a = y
            val b = x
            println("$a, $b")
        }
    }
}

class ProgressTest {
    @Volatile var flag = false
    fun test() {
        thread {
            while (!flag) {}
            println("I am free!")
        }
        thread { flag = true }
    }
}
```

JMM: Volatile fields

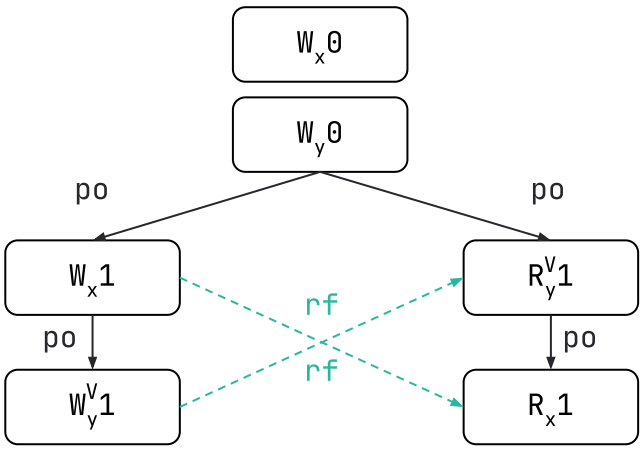
Volatile variables can be used for synchronization.

```
class OrderingTest {
    var x = 0
    @Volatile var y = 0
    fun test() {
        thread {
            x = 1
            y = 1
        }
        thread {
            val a = y
            val b = x
            println("$a, $b")
        }
    }
}
```

How do we know there is enough synchronization?

JMM: Happens-before relation

```
class OrderingTest {
    var x = 0
    @Volatile var y = 0
    fun test() {
        thread {
            x = 1
            y = 1
        }
        thread {
            val a = y
            val b = x
            println("$a, $b")
        }
    }
}
```



po

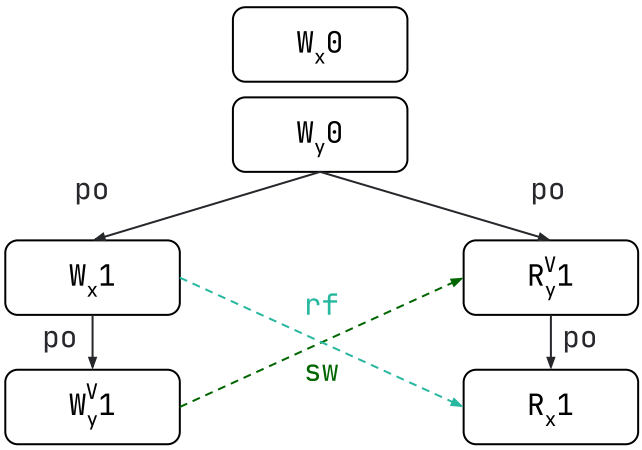
→*program-order*

rf

→*reads-from*

JMM: Happens-before relation

```
class OrderingTest {
    var x = 0
    @Volatile var y = 0
    fun test() {
        thread {
            x = 1
            y = 1
        }
        thread {
            val a = y
            val b = x
            println("$a, $b")
        }
    }
}
```



po

- *program-order*

rf

- *reads-from*

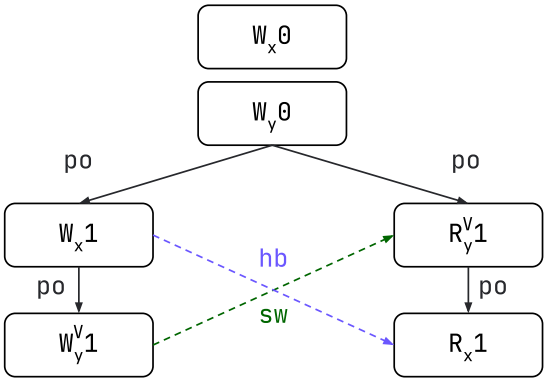
sw

- *Synchronizes-with*

-e.g. reads-from on `Volatile` field

JMM: Happens-before relation

```
class OrderingTest {
    var x = 0
    @Volatile var y = 0
    fun test() {
        thread {
            x = 1
            y = 1
        }
        thread {
            val a = y
            val b = x
            println("$a, $b")
        }
    }
}
```



po

➸*program-order*

rf

➸*reads-from*

sw

➸*Synchronizes-with*

-e.g. reads-from on `Volatile` field

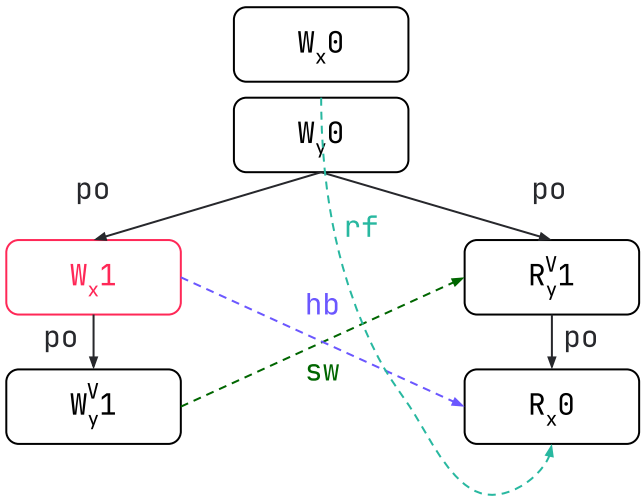
hb

➸*happens-before*

= (po ∪ sw)+

JMM: Happens-before relation

```
class OrderingTest {
    var x = 0
    @Volatile var y = 0
    fun test() {
        thread {
            x = 1
            y = 1
        }
        thread {
            val a = y
            val b = x
            println("$a, $b")
        }
    }
}
```



po — *program-order*

rf — *reads-from*

sw — *Synchronizes-with*
-e.g. reads-from on `Volatile` field

hb — *happens-before*
= (po ∪ sw)⁺

JMM: Synchronizing actions

- Read and write for volatile fields
- Lock and unlock
- Thread run and start, as well as finish and join

JMM: DRF-SC again

Two events form a data race if:

- Both are memory accesses to the same field.
- Both are **plain** (non-atomic) accesses.
- At least one of them is a `write` event.
- They are not related by *happens before.*

Data-race-free programs have **sequentially consistent** semantics

A program is data-race-free if, for every possible execution of this program, no two events form a data race.

JMM: Atomics

But what about atomic operators on shared variables?

```
class Counter {
    private val c = AtomicInteger()

    fun increment() {
        c.incrementAndGet()
    }

    fun decrement() {
        c.decrementAndGet()
    }

    fun value(): Int {
        return c.get()
    }
}
```

JMM: Atomics

Atomic classes from package the `java.util.concurrent.atomic` package:

- `AtomicInteger`
- `AtomicLong`
- `AtomicBoolean`
- `AtomicReference`

And their array counterparts:

- `AtomicIntegerArray`
- `AtomicLongArray`
- `AtomicReferenceArray`

JMM: Atomics

- `get()` – Reads a value with volatile semantics
- `set(v)` – Writes a value with volatile semantics
- `getAndSet(v)` – Atomically exchanges a value
- `compareAndSet(e, v)` – Atomically compares a value of atomic variable with the expected value, e, and if they are equal, replaces content of atomic variable with the desired value, v; returns a boolean indicating success or failure.
- `compareAndExchange(e, v)` – Atomically compares a value with an expected value, e, and if they are equal, replaces with the desired value, v; returns a read value.
- `getAndIncrement()`, `addAndGet(d)`, etc – Perform Atomic arithmetic operations for numeric atomics (AtomicInteger, AtomicLong).
- ...

JMM: Atomics

Methods of atomic classes:

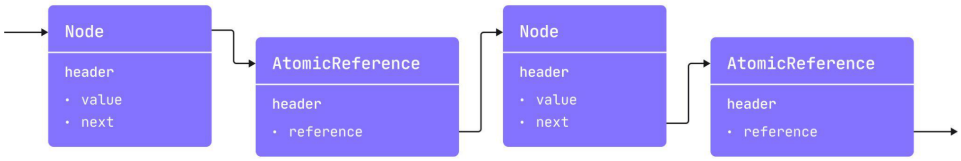
- ...
- `getXXX()`
- `setXXX(v)`
- `weakCompareAndSetXXX(e, v)`
- `compareAndExchangeXXX(e, v)`

In these cases, XXX is an access mode: `Acquire`, `Release`, `Opaque`, `Plain`

You can learn more about Java Access Modes here: https://gee.cs.oswego.edu/dl/html/j9mm.html

JMM: Atomics Problem

```
class Node<T>(val value: T) {
    val next = AtomicReference<Node<T>>()
}
```



JMM: Atomic field updaters

Use `AtomicXXXFieldUpdater` classes to directly modify volatile fields:

```
class Counter {
   @Volatile private var c = 0
   companion object {
       private val updater =
AtomicIntegerFieldUpdater.newUpdater(Counter::class.java, "c")
   }
   fun increment() {
       updater.incrementAndGet(this)
   }
   fun decrement() {
       updater.decrementAndGet(this)
   }
   fun value(): Int {
       return updater.get(this)
   }
}
```

Starting from JDK9, there is also the `VarHandle` class, which serves a similar purpose.

Kotlin: AtomicFU

The AtomicFU library is a recommended way to use atomic operations in Kotlin: https://github.com/Kotlin/kotlinx-atomicfu

```
class Counter {
    private val c = atomic(0)
    fun increment() {
        c += 1
    }
    fun decrement() {
        c -= 1
    }
    fun value(): Int {
        return c.value
    }
}
```

- It provides `AtomicXXX` classes with API similar to Java atomics.
- Under the hood *compiler plugin* replaces usage of atomics to `AtomicXXXFieldUpdater` or `VarHandle`.
- It also provides convenient extension functions, e.g. `c.update { it + 1 }`

JETBRAINS | Kotlin

Thank you

